

Student 18 **Moniga V** 192521184RC

10 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```
numbers = list(map(int, input().split()))

n = len(numbers)

for i in range(n):
    min_index = i

    for j in range(i + 1, n):
        if numbers[j] < numbers[min_index]:
            min_index = j

    numbers[i], numbers[min_index] = numbers[min_index], numbers[i]

print(numbers)
```

INPUT

9 5 1 7

OUTPUT

[1, 5, 7, 9]

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j ≥ 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

arr = list(map(int, input().split()))

sorted_arr = insertion_sort(arr)
print(sorted_arr)
```

INPUT

8 5 7 1 9 3

OUTPUT

[1, 3, 5, 7, 8, 9]

Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

    i = j = k = 0

    # Copy data to temp arrays left_half[] and right_half[]
    while i < len(left_half) and j < len(right_half):
        if left_half[i] < right_half[j]:
            arr[k] = left_half[i]
            i += 1
        else:
            arr[k] = right_half[j]
            j += 1
        k += 1

    while i < len(left_half):
        arr[k] = left_half[i]
        i += 1
        k += 1

    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1

    return arr
arr = list(map(int, input().split()))

# Sort and print the result
sorted_arr = merge_sort(arr)
print(sorted_arr)
```

INPUT

38 27 43 3 9 82 10

OUTPUT

[3, 9, 10, 27, 38, 43, 82]

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q4. Diagonal Matrix

Score: 10.00 TC: 3/3 passed

CODE

```
import json

def is_diagonal_matrix(matrix):
    rows = len(matrix)
    cols = len(matrix[0]) if rows > 0 else 0

    if rows != cols:
        return "Not Diagonal Matrix"

    for i in range(rows):
        for j in range(cols):
            if i != j and matrix[i][j] != 0:
                return "Not Diagonal Matrix"

    return "Diagonal Matrix"

matrix = json.loads(input().strip())

print(is_diagonal_matrix(matrix))
```

INPUT

[[1,0,0],[0,5,0],[0,0,8]]

OUTPUT

Diagonal Matrix

Q5. Row and Column Sums

Score: 6.67 TC: 2/3 passed

CODE

```
raw_input = input().strip()
rows_raw = raw_input.replace(' ', '').strip('[]').split(',')
matrix = [[int(x) for x in row.split(',') for row in rows_raw]

row_sums = [sum(row) for row in matrix]
cols = len(matrix[0]) if matrix else 0
col_sums = [sum(matrix[r][c] for r in range(len(matrix))) for c in range(cols)]

print(f"Row= {row_sums}")
print(f"Col= {col_sums}")
```

INPUT

[[1,2],[3,4]]

OUTPUT

Row= [3, 7]
Col= [4, 6]**Q6. Maximum Subarray Sum**

Score: 6.67 TC: 2/3 passed

CODE

```
nums = list(map(int, input().split()))

max_so_far = nums[0]
curr_max = nums[0]

for x in nums[1:]:
    curr_max = max(x, curr_max + x)
    max_so_far = max(max_so_far, curr_max)

print(max_so_far)
```

INPUT

-2 1 -3 4 -1 2 1 -5 4

OUTPUT

6

Q7. Common Elements Among Three Lists

Score: 10.00 TC: 3/3 passed

CODE

```
list1_str, list2_str, list3_str = input().strip().split(';')

s1 = set(map(int, list1_str.split()))
s2 = set(map(int, list2_str.split()))
s3 = set(map(int, list3_str.split()))

common = sorted(list(s1 & s2 & s3))

print(common)
```

INPUT

1 2 3;2 3 4;2 5

OUTPUT

[2]

Q8. Matrix Multiplication

Score: 6.67 TC: 2/3 passed

CODE

```
raw_input = input().strip()

def parse_matrix(s):
    if '=' in s:
        s = s.split('=', 1)[1].strip()
        s = s.replace(' ', '').strip('[]')
        rows = s.split(',')
        return [[int(x) for x in row.split(',') for row in rows if row]]

if ' ' in raw_input and ('A=' in raw_input or 'B=' in raw_input or '[' in raw_input):
    parts = raw_input.split(' ', 1)
    a_str, b_str = parts[0], parts[1]
else:
    a_str = raw_input
    b_str = input().strip()

A = parse_matrix(a_str)
B = parse_matrix(b_str)

rows_A, cols_A = len(A), len(A[0])
rows_B, cols_B = len(B), len(B[0])

result = []
for i in range(rows_A):
    row = []
    for j in range(cols_B):
        val = sum(A[i][k] * B[k][j] for k in range(cols_A))
        row.append(val)
    result.append(row)

if len(result) == 1:
    print(f"[{' '.join(map(str, result[0]))}]")
else:
    rows_formatted = [f"[{' '.join(map(str, row))}]" for row in result]
    print("[\n ".join(rows_formatted) + "\n"])
```

INPUT

A=[[1,2],[3,4]] B=[[5,6],[7,8]]

OUTPUT[[19, 22]
[43, 50]]

Q9. Sort Rotate Min Max

Score: 3.33 TC: 1/3 passed

CODE

```
raw_input = input().strip()

parts = [p.strip() for p in raw_input.split(' ')]

list_str = parts[0]
k_str = parts[1]

if '=' in list_str:
    list_str = list_str.split('=', 1)[1]

if '=' in k_str:
    k_str = k_str.split('=', 1)[1]

nums = [int(x) for x in list_str.strip().split()]
k = int(k_str.strip())

sorted_nums = sorted(nums)

n = len(sorted_nums)
k = k % n if n > 0 else 0
rotated_nums = sorted_nums[k:] + sorted_nums[:k]

min_val = min(sorted_nums)
max_val = max(sorted_nums)

print(f"Sorted= {sorted_nums}")
print(f"Rotated= {rotated_nums}")
print(f"Min= {min_val}")
print(f"Max= {max_val}")
```

INPUT

List=5 2 8 1 4,k=2

OUTPUT

```
Sorted= [1, 2, 4, 5, 8]
Rotated= [4, 5, 8, 1, 2]
Min= 1
Max= 8
```

Q10. Sort Matrix Search

Score: 3.33 TC: 1/3 passed

CODE

```
raw_input = input().strip()

if 'Find=' in raw_input:
    matrix_str, target_str = raw_input.split('Find=', 1)
elif 'find=' in raw_input:
    matrix_str, target_str = raw_input.split('find=', 1)
else:
    idx = raw_input.rfind(',')
    matrix_str, target_str = raw_input[:idx], raw_input[idx+1:]

matrix_str = matrix_str.rstrip(' ')
if '=' in matrix_str:
    matrix_str = matrix_str.split('=', 1)[1]

target = int(target_str.strip(' ,='))

matrix_str = matrix_str.replace(' ', '').strip('[]')
rows = matrix_str.split(',')
matrix = [[int(x) for x in row.split(',') for row in rows if row]]

r_count = len(matrix)
c_count = len(matrix[0]) if r_count > 0 else 0

flat = sorted([val for row in matrix for val in row])

sorted_matrix = []
for i in range(r_count):
    sorted_matrix.append(flat[i * c_count : (i + 1) * c_count])

pos = None
for r in range(r_count):
    for c in range(c_count):
        if sorted_matrix[r][c] == target:
            pos = (r, c)
            break
    if pos:
        break

if pos:
    print(f"Sorted= {sorted_matrix}")
    print(f"Position= {pos}")
else:
    print("Not Found")
```

INPUT

Matrix=[[9,3],[7,1]], Find=7

OUTPUTSorted= [[1, 3], [7, 9]]
Position= (1, 0)

Q9. Sort Rotate Min Max

Score: 0.00 TC: 0/3 passed

CODE

```
s = input().strip()

p = s.split(",k=")

a = p[0].replace("List=", "").strip()
k = int(p[1])

a = list(map(int, a.split()))

for i in range(len(a)):
    m = i
    for j in range(i + 1, len(a)):
        if a[j] < a[m]:
            m = j
    a[i], a[m] = a[m], a[i]

k = k % len(a)

if k == 0:
    rotated = a
else:
    rotated = a[-k:] + a[:-k]

print("Sorted=", a)
print("Rotated=", rotated)
print("Min=", min(rotated))
print("Max=", max(rotated))
```

INPUT

List=9 7 5,k=3

OUTPUT

```
Sorted= [5, 7, 9]
Rotated= [5, 7, 9]
Min= 5
Max= 9
```